German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026

Milestone 1: Event Ticketing Platform

Core Services Foundation

Deadline is Saturday 11/04/2026 at 11:59 PM

Contents

1 Overview

This milestone is worth **15%** of your final grade. You will build an **Event Ticketing Platform** consisting of 5 independently runnable Spring Boot services organized as a **Maven multi-module project**. All 5 services connect to a single shared PostgreSQL database and are orchestrated via a single `docker-compose.yaml`.

In future milestones, these services will be refactored into true microservices with separate databases, inter-service communication (OpenFeign, RabbitMQ, etc.), and Kubernetes deployment. For now, they are independent applications sharing one database.

Technologies: Spring Boot, PostgreSQL, Spring Data JPA, Docker, Docker Compose, Maven multi-module, Git & GitHub.

Services:

a) **User Service** (port 8081) — Attendee accounts, profiles, preferences

b) **Event Service** (port 8082) — Event catalog, sessions, venues

c) **Booking Service** (port 8083) — Booking lifecycle, line items, reservations

d) **Ticket Service** (port 8084) — Ticket issuance, validation, tracking

e) **Sales Service** (port 8085) — Payment processing, refunds, promotions

Deliverables: 45 features (9 per service), full CRUD for all entities, Dockerized multi-service application, Git history with personalized feature branches and PRs.

2 Personalization & Team Roster

Every team member's work must be **individually identifiable** in Git history. The auto-grader cross-references the `team.json` roster against `git log` to determine who built what. Violations result in **zero credit** for the affected member.

2.1 team.json (Required)

Create a file named `team.json` in the **project root directory** (next to the parent `pom.xml`). It contains a JSON array with one entry per team member:

```
[{"studentId":"55-8078","name":"Ahmed
Ali","githubUsername":"ahmed-ali-dev","service":"user-service"}, ...]
```

Fields per entry:

- `studentId` — University ID in the format `XX-XXXX`
- `name` — Full name
- `githubUsername` — The exact GitHub username that will author commits
- `service` — Which service module this member belongs to (one of: `user-service`, `event-service`, `booking-service`, `ticket-service`, `sales-service`)

The auto-grader reads this file and then runs specific tests to find each member's commits. It matches those commits against branch names to determine which features each member implemented. **Do not self-declare features** — the grader discovers them automatically.

2.2 Branch Naming (Mandatory)

All feature branches must include the implementing member's student ID:

```
feat/<service>/<feature-name>/<studentId>
```

Example: student `55-8078` implementing S1-F1 (Service 1, Feature 1) creates:

```
feat/user/S1-F1/55-8078
```

2.3 Commit Message Format (Mandatory)

All commits must follow this convention:

```
feat(<service name>):  <description> (ID)
```

Examples: `feat(user-service):  add User entity model (55-8078)`, `fix(booking-service):  fix null handling in search query (55-8078)`.

The auto-grader matches each commit's Git author (from `team.json`) against the student ID in the message. Mismatched or missing IDs will result in failures during grading. So even if you named the commit message wrong or the branch name was wrong and did follow the criteria, you know how to fix it using the commands that you took in the Git Lab.

3 Project Structure — Multi-Module Maven

The project is organized as a Maven multi-module project. The root directory contains a **parent POM** with `<packaging>pom</packaging>` that declares all 5 services as modules. Each service is a fully independent Spring Boot application with its own POM, source tree, and `@SpringBootApplication` entry point. Section 4 provides full step-by-step instructions for creating this structure.

3.1 Port & Database Configuration

All services connect to the **same PostgreSQL database** (`eventticketingdb`). Each runs on a different port:

Service	Internal Port	Docker Host Port
user-service	8080	8081
event-service	8080	8082
booking-service	8080	8083
ticket-service	8080	8084
sales-service	8080	8085

Each service's `application.properties` sets `server.port=8080`. The port differentiation happens in `docker-compose.yaml` via host-port mapping (e.g., `8081:8080` for user-service). When running services locally without Docker during development, each developer works on their own service and can use port 8080 since they only run one service at a time.

Each service configures: `spring.datasource.url=jdbc:postgresql://localhost:5432/eventticketingdb` (or `postgres:5432` inside Docker), `ddl-auto=update`, and `show-sql=true`.

4 Step-by-Step: Creating the Multi-Module Project

This section walks you through creating the entire project structure from scratch. Two approaches are provided: **Option A** using IntelliJ IDEA (recommended), and **Option B** using Spring Initializr for students not using IntelliJ. Both produce the same result. Follow **one** of the two options, then continue with the shared steps.

Important architectural note: The root project acts as an **aggregator only**. It does not contain source code and does not act as a Maven parent for the child modules. Each service module keeps its own generated Spring Boot parent. The root `pom.xml` simply lists the modules so you can build them all with a single command.

4.1 Using IntelliJ IDEA (Recommended)

4.1.1 A1. Create the Parent Project

a) Open IntelliJ IDEA → **File** → **New** → **Project**.

b) On the left panel, select **Java**.

c) On the right, set **Build system** to **Maven**.

d) Set the **JDK** to **25**.

e) Uncheck **Add sample code**.

f) Expand **Advanced Settings** (at the bottom of the dialog).

g) Fill in:

- **Name:** `eventticketing`
- **GroupId:** `com.teamXX.eventticketing` (replace `XX` with your team number)
- **ArtifactId:** `eventticketing`

h) Click **Create**.

i) IntelliJ creates a Maven project with a `pom.xml`. Open this `pom.xml` and manually add the following line inside the `<project>` block (after `<version>`):

```
<packaging>pom</packaging>
```

j) If IntelliJ created a `src/` folder in the root project, **delete it** — the root aggregator project has no source code.

k) **Reload Maven:** Click the Maven reload icon (circular arrows) in the Maven tool window, or right-click the `pom.xml` → **Maven** → **Reload project**.

4.1.2 A2. Add Each Service as a Module

Repeat the following for each of the 5 services (`user-service`, `event-service`, `booking-service`, `ticket-service`, `sales-service`):

a) Right-click the project root (`eventticketing`) in the Project panel → **New** → **Module**.

b) Select **Spring Boot** on the left.

c) Configure the service information:

- **Name / Artifact:** the service name (e.g., `user-service`)

- **Group:** `com.teamXX.eventticketing`
- **Package name:** Use the service domain name **without hyphens**:
  - `com.teamXX.eventticketing.user`
  - `com.teamXX.eventticketing.event`
  - `com.teamXX.eventticketing.booking`
  - `com.teamXX.eventticketing.ticket`
  - `com.teamXX.eventticketing.sales`
- **Type / Build system:** Maven
- **Java:** 25
- **Packaging:** Jar

Warning: The `artifactId` can use hyphens (e.g., `user-service`), but **Java package names cannot contain hyphens**. Use `.user`, not `.user-service`, in the package name.

d) Click **Next** to continue to the dependencies page.

e) Select these dependencies: **Spring Web**, **Spring Boot DevTools**, **Spring Data JPA**, **PostgreSQL Driver**, **Rest Repositories**.

f) Click **Create**.

4.1.3 A3. What IntelliJ Does (and Does Not Do) Automatically

After creating each service module, IntelliJ will usually:

- Create the service folder with its own `pom.xml` and `src/` tree.
- Generate the `@SpringBootApplication` main class.

However, IntelliJ may **not** automatically add the `<module>` entry to the root `pom.xml`. You must verify this yourself after each module creation.

4.1.4 A4. Verify and Complete the Root POM

After creating all 5 service modules:

a) Open the root `eventticketing/pom.xml`.

b) Confirm that `<packaging>pom</packaging>` exists.

c) Check whether all 5 services appear inside a `<modules>` block. If **any are missing**, add them manually:

```
<modules>
    <module>user-service</module>
    <module>event-service</module>
    <module>booking-service</module>
    <module>ticket-service</module>
    <module>sales-service</module>
</modules>
```

d) Save the file.

e) **Reload Maven** to pick up the changes.

f) Confirm there are no Maven errors in the tool window.

4.1.5 A5. Important: Do NOT Modify Child Service POMs

Each generated service already has its own `<parent>` block pointing to Spring Boot. **Do not replace or modify this parent.** The root `eventticketing` POM acts only as an **aggregator** (it lists the modules so you can build them all at once). It is **not** a Maven parent for the child modules.

In other words:

- **Do not** add a `<parent>` block pointing to `eventticketing` inside any child POM.
- **Do not** remove the existing Spring Boot `<parent>` from any child POM.
- The only connection between the root POM and the children is the `<modules>` block in the root.

4.1.6 Add jackson-databind Dependency

In each service's `pom.xml`, add the following dependency inside the `<dependencies>` block (required for Hibernate JSONB support):

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
</dependency>
```

After adding, **reload Maven**.

4.1.7 Configure application.properties per Service

Inside each service's `src/main/resources/application.properties`, add:

```
server.port=8080

spring.datasource.url=jdbc:postgresql://localhost:5432/eventticketingdb
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=
                org.hibernate.dialect.PostgreSQLDialect
```

All services use port 8080 internally. The port differentiation to 8081–8085 happens in `docker-compose.yaml` (covered in the Dockerization section).

4.1.8 Create Sub-Packages

Inside each service's main package (e.g., `src/main/java/com/teamXX/eventticketing/user/`), create these sub-packages: `model/`, `repository/`, `service/`, `controller/`, `dto/`.

4.1.9 Add a Health Endpoint to Each Service

In each service's `controller/` package, create a simple health controller that exposes a GET endpoint returning `"OK"` (200):

- User Service: `GET /api/users/health`
- Event Service: `GET /api/events/health`
- Booking Service: `GET /api/bookings/health`

- Ticket Service: `GET /api/tickets/health`
- Sales Service: `GET /api/sales/health`

4.1.10 Create team.json

In the project root (next to the root `pom.xml`), create `team.json` as described in Section 2.

4.1.11 Verify the Build

Start PostgreSQL via Docker:

```
docker compose up -d
```

(This requires the `docker-compose.yaml` with just the PostgreSQL service — see Section 6.)
Then build all services from the project root:

```
mvn clean package -DskipTests
```

If the build succeeds, you should see `BUILD SUCCESS` and a JAR in each service's `target/` directory.
To test a single service locally:

```
cd user-service
mvn spring-boot:run
```

Then visit `http://localhost:8080/api/users/health` to confirm it responds with `"OK"`.

4.2 Final Project Structure

After completing all steps, your project should look like this:

```
eventticketing/
+-- pom.xml                              (root aggregator POM, packaging=pom)
+-- team.json                            (team roster)
+-- docker-compose.yaml                  (PostgreSQL, later: all services)
+-- .gitignore
+-- user-service/
|   +-- pom.xml                          (own Spring Boot parent, NOT eventticketing)
|   +-- Dockerfile                       (added in Phase D)
|   +-- src/main/java/com/teamXX/eventticketing/user/
|       +-- UserApplication.java         (@SpringBootApplication)
|       +-- model/
|       +-- repository/
|       +-- service/
|       +-- controller/
|       +-- dto/
+-- event-service/
|   +-- (same structure)
+-- booking-service/
|   +-- (same structure)
+-- ticket-service/
|   +-- (same structure)
+-- sales-service/
    +-- (same structure)
```

5 Git Workflow — Initial Setup

a) Initialize a Git repository in the project root.

b) Create a `.gitignore` excluding: `target/`, `.idea/`, `*.iml`, `.env`, `*.class`, `.DS_Store`.

c) Create the parent POM, all 5 child POM files, the package structure, and the `team.json` file.

d) Make an initial commit: `init: project setup by (Name_ID)` (e.g: init: project setup by (Mohamed_Ayman_52_8078)) (note: this is supposed to be the only commit message that will not follow the commit message rule that is stated in this description).

e) Create a **private** GitHub repository named `TeamNumber-TeamName-EventTicketing` (e.g: 40-Random-EventTicketing).

f) Push to remote and rename the default branch to `main`.

g) **Branch protection:** Enable *"Require a pull request before merging"* on `main`. (The rule may not be enforced on the free version of git so you have to enforce it manually as the grader will check the generated PRs)

h) Add `Scalable-Submissions` as a collaborator.

i) **Each member** must contribute to the project through Git using their own GitHub account. The auto-grader verifies all GitHub usernames from `team.json` appear in history of the repo. ***If someone didn't contribute to the project (no-commits for him/her), will be considered as if they didn't work at all in the milestone and will receive a ZERO as a result in the milestone grade.***

6 Database Setup (Docker Compose)

The `docker-compose.yaml` in the project root contains a PostgreSQL service with:

- Container name: `eventticketing-db`
- Port mapping: `5432:5432`
- Environment variables: user `postgres`, password `postgres`, database `eventticketingdb`
- Named volume `pgdata` for data persistence

In Phase D (Dockerization), you will add all 5 application services to this same file.

Verify: Run `docker compose up -d` (PostgreSQL only at first), start any service locally, and confirm it connects to the database.

7 Entity Models

All entities use auto-generated `Long` IDs. Some entities include one JSONB column implemented as a `Map<String, Object>` using the Hibernate JSONB annotations learned in Lab 4. Since all services share one database, tables from different services coexist in the same database schema.

Each service defines JPA relationships (`@OneToMany`, `@ManyToOne`, join entities) **between entities within the same service only**. References to entities in other services are stored as plain `Long` foreign key columns (not JPA-managed relationships), and cross-service data access uses native SQL JOINs on the shared database. Remember to use `@JsonIgnore` on bidirectional relationships to prevent infinite recursion during JSON serialization.

7.1 User Service Entities (`user-service`)

7.1.1 User Entity

Table: `users`

Field	Type	Constraints	Notes
`id`	`Long`	PK, auto-generated	
`name`	`String`	not null	
`email`	`String`	not null, unique	
`password`	`String`	not null	
`phone`	`String`	not null, unique	
`role`	`ENUM`[a]	not null	`"ATTENDEE"` or `"ADMIN"`
`status`	`ENUM`	not null, default ACTIVE	`ACTIVE / DEACTIVATED`
`preferences`	`Map<String, Object>`	JSONB	See below
`createdAt`	`LocalDateTime`	not null	Set on creation
`favoriteVenues`	`List<FavoriteVenue>`	–	Relational Attribute

[a]search about how you would save an enum inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — `preferences`: Language, notification settings (email/sms booleans), favorite event categories, preferred ticket type (VIP/standard), timezone.

Relationships: *One* User can have *Many* FavoriteVenues and ***User*** is the inverse side and ***FavoriteVenue*** is the owner side.

7.1.2 FavoriteVenue Entity

Table: `favorite_venues`

Field	Type	Constraints	Notes
`id`	`Long`	PK, auto-generated	
`label`	`String`	not null	e.g., "My Go-To", "Weekend Spot"
`venueName`	`String`	not null	e.g., "Cairo Opera House"
`location`	`String`	not null	e.g., "Zamalek, Cairo"
`capacity`	`Integer`	nullable	Venue capacity if known
`isDefault`	`Boolean`	default false	Home venue
`metadata`	`Map<String, Object>`	JSONB	See below
`createdAt`	`LocalDateTime`	not null	
`user`	`User`	–	Relational Attribute

JSONB — `metadata`: Parking availability, accessibility (wheelchair, elevators), food options, website URL, nearby transport, personal notes.

Relationships: *Many* FavoriteVenues can belong to *One* User and ***User*** is the inverse side and ***FavoriteVenue*** is the owner side.

7.2 Event Service Entities (`event-service`)

7.2.1 Event Entity

Table: `events`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
name	String	not null	
venue	String	not null	
eventDate	LocalDateTime	not null	
category	ENUM[a]	not null	See values below
status	ENUM	not null	UPCOMING / ONGOING / COMPLETED / CANCELLED
rating	Double	default 0.0	Running average
totalRatings	Integer	default 0	Count for average calculation
details	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
eventSessions	List<EventSession>	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

Category values: CONCERT, SPORTS, THEATER, CONFERENCE, FESTIVAL.

JSONB — details: Organizer name, venue capacity, ticket price range, age restriction, description, sponsors (list), venue latitude, venue longitude.

Relationships: *One* Event has *Many* EventSessions and ***Event*** is the inverse side and ***EventSession*** is the owner side.

7.2.2 EventSession Entity

Table: event_sessions

An event can have multiple sessions (e.g., a 3-day conference has a morning session, afternoon session each day, or a festival has Day 1, Day 2).

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
title	String	not null	e.g., "Day 1 - Opening"
speaker	String	nullable	Presenter or performer
startTime	LocalDateTime	not null	
endTime	LocalDateTime	not null	
capacity	Integer	not null	Max attendees for this session
verified	Boolean	default false	Approved by organizer
metadata	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
event	Event	–	Relational Attribute

JSONB — metadata: Room name, equipment needed (list), language, accessibility (wheelchair, sign language), recording available (boolean).

Relationships: *Many* EventSessions belong to *One* Event and ***Event*** is the inverse side and ***EventSession*** is the owner side.

7.3 Booking Service Entities (booking-service)

7.3.1 Booking Entity

Table: bookings

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
userId	Long	not null	FK reference (not JPA-managed)
eventId	Long	nullable	FK reference (assigned later)
contactEmail	String	not null	
status	ENUM[a]	not null	See values below
totalAmount	Double	nullable	Set when calculated
metadata	Map<String, Object>	JSONB	See below
bookingDate	LocalDateTime	not null	
confirmedAt	LocalDateTime	nullable	Set when confirmed
bookingItems	List<BookingItem>	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

Status values: PENDING, CONFIRMED, CHECKED_IN, COMPLETED, CANCELLED.

JSONB — metadata: Ticket tier (VIP, standard, early-bird), special requests (wheelchair, dietary), group size, referral code, notes.

Note: userId and eventId are plain Long columns, **not** JPA @ManyToOne relationships (User and Event live in different services).

Relationships: *One* Booking can have *Many* Items and ***Booking*** is the inverse side while ***BookingItem*** is the owner side.

7.3.2 BookingItem Entity

Table: booking_items

A booking can contain multiple items (e.g., 2 VIP tickets + 3 standard tickets for the same event). Each item has a line number within the booking.

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
eventOrder	Integer	not null	1, 2, 3...
sessionId	Long	not null	FK reference (not JPA-managed)
sessionTitle	String	not null	Snapshot at booking time
quantity	Integer	not null	Number of seats
unitPrice	Double	not null	Price per seat
status	ENUM[a]	not null	RESERVED / CONFIRMED / REFUNDED
metadata	Map<String, Object>	JSONB	See below
booking	Booking	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — metadata: Seat section, row number, ticket tier, attendee names (list), notes.

Relationships: *Many* Items can belong to *One* Booking and ***Booking*** is the inverse side while ***BookingItem*** is the owner side.

7.4 Ticket Service Entities (ticket-service)

7.4.1 Ticket Entity

Table: tickets

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
bookingId	Long	not null	FK reference (not JPA-managed)
attendeeName	String	not null	
ticketCode	String	not null, unique	Generated code
status	ENUM[a]	not null	VALID / USED / EXPIRED / CANCELLED
issuedAt	LocalDateTime	not null	
metadata	Map<String, Object>	JSONB	See below

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — `metadata`: Seat number, section, tier (VIP/standard), gate, QR code URL, check-in time.

Relationships: None within this service. `bookingId` is a plain `Long` referencing the `bookings` table in the shared database via native SQL.

7.5 Sales Service Entities (`sales-service`)

This service demonstrates the **join entity pattern** for many-to-many relationships with extra columns (like the `OrderItem` pattern from Lab 4).

7.5.1 TicketSale Entity

Table: `ticket_sales`

Field	Type	Constraints	Notes
id	Long	PK, auto-generated	
bookingId	Long	not null	FK reference (not JPA-managed)
userId	Long	not null	FK reference (not JPA-managed)
amount	Double	not null	
method	ENUM[a]	not null	CREDIT_CARD / DEBIT_CARD / WALLET
status	ENUM	not null	See below
transactionDetails	Map<String, Object>	JSONB	See below
createdAt	LocalDateTime	not null	
salePromotions	List<SalePromotion>	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

Status values: `PENDING`, `COMPLETED`, `FAILED`, `REFUNDED`.

JSONB — `transactionDetails`: Gateway response, card last four digits, receipt URL, failure reason, booking reference.

Relationships: ***One*** TicketSale can reference ***Many*** SalePromotions and ***TicketSale*** is the inverse side while ***SalePromotion*** is the owner side.

7.5.2 Promotion Entity

Table: `promotions`

Field	Type	Constraints	Notes
`id`	`Long`	PK, auto-generated	
`code`	`String`	not null, unique	e.g., `"SHOW25"`
`discountType`	`ENUM`[a]	not null	`PERCENTAGE` or `FIXED`
`discountValue`	`Double`	not null	e.g., 25.0
`maxUses`	`Integer`	not null	
`currentUses`	`Integer`	default 0	
`expiryDate`	`LocalDateTime`	not null	
`active`	`Boolean`	default true	
`metadata`	`Map<String, Object>`	JSONB	See below
`salePromotions`	`List<SalePromotion>`	–	Relational Attribute

[a]search about how you would save an enum ***as an ENUM*** inside SQL and what are the necessary configurations that you have to do for JPA

JSONB — `metadata`: Eligible event categories, minimum booking amount, terms and conditions, applicable event IDs.

Relationships: ***One*** Promotion can reference ***Many*** SalePromotions and ***Promotion*** is the inverse side while ***SalePromotion*** is the owner side.

7.5.3 SalePromotion Entity (Join Entity)

Table: `sale_promotions`

This is a **join entity** linking `TicketSale` and `Promotion` in a many-to-many relationship with extra columns — the same pattern as `OrderItem` from Lab 4. A ticket sale can use multiple promotions, and a promotion can be used on multiple ticket sales.

Field	Type	Constraints	Notes
`id`	`Long`	PK, auto-generated	
`discountApplied`	`Double`	not null	Actual amount deducted
`appliedAt`	`LocalDateTime`	not null	When promotion was applied
`ticketSale`	`TicketSale`	–	Relational Attribute
`promotion`	`Promotion`	–	Relational Attribute

Relationships:

- *Many* SalePromotions can belong to *One* TicketSale and ***TicketSale*** is the inverse side while ***SalePromotion*** is the owner side.
- *Many* SalePromotions can belong to *One* Promotion and ***Promotion*** is the inverse side while ***SalePromotion*** is the owner side.

7.6 Relationship Summary

Service	Relationship	Pattern Tested
User	User ↔ FavoriteVenue	Bidirectional
Event	Event ↔ EventSession	Bidirectional
Booking	Booking ↔ BookingItem	Bidirectional + ordering [a]
Ticket	(no JPA relations)	Cross-service via native SQL
Sales	TicketSale ↔ Promotion	Join entity (SalePromotion)

[a]You will use the ordering in a specific feature from the feature list

7.7 Important: Cross-Service Data Access

Since all services share one PostgreSQL database, any service can query any table using **native SQL**. For example, the Booking Service can `JOIN bookings` with `events` or `ticket_sales` in a native `@Query`. However, each service only defines JPA entity classes and JPA relationships for entities within its *own* module. Cross-table access is always via native SQL, never via JPA relationships. This pattern will be replaced by Micro-services Communications in Milestone 3.

8 Repository Layer

Each service defines a `JpaRepository` for **each of its entities**. For services with multiple entities, you need multiple repository interfaces. ***Make sure that you make use of the Repository Layer when interacting with the database and not have the queries at the service layer as the auto grader will test having proper layering logic in your code.*** Repositories include:

- **Naming-convention methods** for simple lookups.
- **Custom `@Query` methods with native SQL** for complex queries, including JOINs against other services' tables in the shared database.
- `@Modifying` + `@Transactional` for UPDATE/DELETE queries.

9 Features

CRUD operations (create, read by ID, read all, update, delete) are the **baseline** for **every entity in every service** and are NOT counted as features. ***However this doesn't mean that you don't need to implement it as it will have test cases and the auto grader will make use of it and will not run if they are not there so*** This means: User Service needs CRUD for both User and FavoriteVenue, Event Service for both Event and EventSession, Booking Service for both Booking and BookingItem, Ticket Service for Ticket, and Sales Service for TicketSale, Promotion, and SalePromotion. Implement all CRUD first before starting features.

Important Note: Do not forget to apply the Layered architecture (having repository, service and controller) as the test case will test that each feature is implemented according to the layered architecture as explained to you in the Labs.

9.1 User Service Features (port 8081)

9.1.1 [S1-F1] Search Users with Filters

Branch: `feat/user/S1-F1/<ID>`
Endpoint: `GET /api/users/search?name={name}&email={email}&role={role}`
Response: List of users matching any non-null filter. Name and email support partial matching (case-insensitive). Return empty list if no matches.
Note: You may use Native SQL query but you will have to find a way to handle null parameters.

Test scenario:

a) Create 3 users via CRUD: one with name "Ahmed", role "ATTENDEE"; one with name "Sara", role "ADMIN"; one with name "Ahmed Ali", role "ATTENDEE".

b) `GET /api/users/search?name=Ahmed` → should return 2 users.

c) `GET /api/users/search?role=ADMIN` → should return only "Sara".

d) `GET /api/users/search?name=xyz` → should return empty list.

e) `GET /api/users/search?name=ahmed` → should still return 2 (case-insensitive).

9.1.2 [S1-F2] Update User Preferences (JSONB)

Branch: `feat/user/S1-F2/<ID>`
Endpoint: `PUT /api/users/{id}/preferences`
Request body: JSON object of preference key-value pairs.
Behavior: Find user –***throws 404 if user not found***. **Merge** incoming preferences into existing map—do not overwrite. The only case that you overwrite the preferences is that if the same keys are already there in the preferences, otherwise if it's a new preference then add it to the current preferences. Return updated user.

Test scenario:

a) Create a user with preferences `{"language":"en","favoriteCategory":"CONCERT"}`.

b) `PUT /api/users/{id}/preferences` with body `{"favoriteCategory":"SPORTS","ticketTier":"VIP"}`.

c) Verify response has `{"language":"en","favoriteCategory":"SPORTS","ticketTier":"VIP"}` — "language" kept, "favoriteCategory" updated, "ticketTier" added.

d) `PUT /api/users/999/preferences` → should return 404.

9.1.3 [S1-F3] Get User Booking Summary (DTO)

Branch: `feat/user/S1-F3/<ID>`
Endpoint: `GET /api/users/{id}/booking-summary`
Response DTO (`UserBookingSummaryDTO`)**:** `userId`, `name`, `totalBookings`, `completedBookings`, `cancelledBookings`, `totalSpent`, `averageBookingAmount`.
Behavior: Find user –***throws 404 if user not found***. Aggregate data from the `bookings` table for this user using native SQL JOINs on the shared database. Build and return DTO.[1]

Test scenario:

a) Create a user. Create 5 bookings: 3 COMPLETED with amounts 300, 500, 700; 1 CANCELLED; 1 PENDING.

b) `GET /api/users/{id}/booking-summary` → should return: totalBookings=5, completedBookings=3, cancelledBookings=1, totalSpent=1500, averageBookingAmount=500.

c) Test with a user who has no bookings → all values should be 0.

d) Test with non-existent user ID → 404.

9.1.4 [S1-F4] Deactivate User Account

Branch: `feat/user/S1-F4/<ID>`
Endpoint: `PUT /api/users/{id}/deactivate`
Behavior (Transactional): Find user –***throws 404 if user not found***. Check no active bookings exist for this user in the `bookings` table –***throw 400 if the user has active bookings***. Set `status` to `"DEACTIVATED"`. Save.

Test scenario:

a) Create a user. Create a booking with status PENDING for that user.

b) `PUT /api/users/{id}/deactivate` → should return 400 (active booking exists).

c) Update the booking status to COMPLETED.

d) `PUT /api/users/{id}/deactivate` → should succeed. Verify user status is now "DEACTIVATED".

[1] Since you will use native SQL here, you can construct the DTO manually from the Object[] that will be returned from the query

9.1.5 [S1-F5] Filter Users by Preference (JSONB Query)

Branch: `feat/user/S1-F5/<ID>`
Endpoint: `GET /api/users/preferences/search?key={key}&value={value}`
Behavior: Validate key/value not blank –***throws 400 if they are***. Return users whose `preferences` matches key-value pair.

Test scenario:

a) Create 3 users: one with preferences `{"favoriteCategory":"CONCERT"}`
, one with `{"favoriteCategory":"SPORTS"}`, one with `{"favoriteCategory":"CONCERT"}`.

b) `GET /api/users/preferences/search?key=favoriteCategory&value=CONCERT` → should return 2 users.

c) `GET /api/users/preferences/search?key=favoriteCategory&value=THEATER` → empty list.

d) `GET /api/users/preferences/search?key=&value=CONCERT` → should return 400.

9.1.6 [S1-F6] Top Attendees by Spending (Report DTO)

Branch: `feat/user/S1-F6/<ID>`
Endpoint: `GET /api/users/reports/top-attendees?startDate={date}&endDate={date}&limit={n}`
Response DTO (`TopAttendeeDTO`): `userId`, `name`, `totalSpent`, `bookingCount`.
Behavior: The feature means show me the top **N** attendees who spent the most money on event bookings in a given time period. Validate dates –***throws 400 if not valid (if start after end)***. Build the query and construct a DTO from the result of the query in the same way as you did in S1-F3.

Test scenario:

a) Create 3 users. Create completed bookings: User A = 2000 total, User B = 5000 total, User C = 800 total, all within March 2026.

b) `GET /api/users/reports/top-attendees?startDate=2026-03-01&endDate=2026-03-31&limit=2` → should return User B first, then User A.

c) Test with dates where no bookings exist → empty list.

d) Test with startDate after endDate → 400.

9.1.7 [S1-F7] Set Default Favorite Venue (Transactional + Relationship)

Branch: `feat/user/S1-F7/<ID>`
Endpoint: `PUT /api/users/{userId}/venues/{venueId}/default`
Behavior (Transactional): Find user –***throws 404 if user not found***. Find the FavoriteVenue by its ID –***throws 404 if not found***. Verify the venue belongs to this user –***throws 400 if it doesn't belong to that user***. Set `isDefault = false` on all of the user's existing favorite venues. Set `isDefault = true` on the target venue. Return the updated user with their favorite venues.

Test scenario:

a) Create a user with 3 favorite venues. Set venue #2 as default.

b) `PUT /api/users/{userId}/venues/{venue3Id}/default` → verify venue #3 is now default and #2 is no longer default.

c) Test with non-existent user → 404.

d) Test with a venue ID that belongs to a different user → 400.

9.1.8 [S1-F8] Get User Profile with Favorite Venues (Relationship DTO)

Branch: `feat/user/S1-F8/<ID>`
Endpoint: `GET /api/users/{id}/profile`
Response DTO (`UserProfileDTO`): `userId`, `name`, `email`, `phone`, `preferences` (JSONB), `favoriteVenues` (list of venue objects with label, venueName, location, capacity, isDefault, metadata), `totalFavoriteVenues` (count).
Behavior: Find user with their favorite venues loaded —***throws (404 if user not found)***. Build the DTO from the user entity and its venue collection. The DTO must include the full list of venues, not just IDs.

Test scenario:

a) Create a user with preferences and 3 favorite venues (one marked default).

b) `GET /api/users/{id}/profile` → verify DTO contains user info, preferences, full venue list with all fields, and totalFavoriteVenues = 3.

c) Create a user with no favorite venues → verify favoriteVenues is empty list and totalFavoriteVenues = 0.

d) Test with non-existent user → 404.

9.1.9 [S1-F9] Find Users by Favorite Category with Minimum Bookings (JSONB + SQL)

Branch: `feat/user/S1-F9/<ID>`
Endpoint: `GET /api/users/preferences/category?category={cat}&minBookings={n}`
Behavior: This feature is as if you want to find all users who prefer category **x** and have made at least **n** completed bookings. Validate category not blank –***throws 400 if blank***.

Test scenario:

a) Create 3 users: User A (favoriteCategory=CONCERT, 5 completed bookings), User B (favoriteCategory=CONCERT, 2 completed bookings), User C (favoriteCategory=SPORTS, 10 completed bookings).

b) `GET /api/users/preferences/category?category=CONCERT&minBookings=3` → should return only User A.

c) `GET /api/users/preferences/category?category=CONCERT&minBookings=1` → should return User A and User B.

d) `GET /api/users/preferences/category?category=&minBookings=1` → 400.

9.2 Event Service Features (port 8082)

9.2.1 [S2-F1] Search Events by Category and Date Range

Branch: `feat/event/S2-F1/<ID>`
Endpoint: `GET /api/events/search?category={c}&startDate={d}&endDate={d}`
Behavior: Returns a list of events matching the optional category and within the date range on `eventDate`. Order by eventDate ascending (soonest first).

Test scenario:

a) Create 3 events: CONCERT on March 15, SPORTS on March 20, CONCERT on March 25.

b) `GET /api/events/search?category=CONCERT&startDate=2026-03-01&endDate=2026-03-31` → returns 2, March 15 first.

c) `GET /api/events/search?startDate=2026-03-01&endDate=2026-03-31` → returns all 3 (no category filter).

9.2.2 [S2-F2] Update Event Details (JSONB Partial Update)

Branch: `feat/event/S2-F2/<ID>`
Endpoint: `PUT /api/events/{id}/details`
Request body: JSON object of detail fields to update.
Behavior: Find event –***throws 404 if not found***. Merge incoming fields into existing `details` map. meaning if different attributes were sent in the json body, then add it to the existing one, else then you will update it. For example, if the event's current details is:

```
{"organizer": "LiveNation", "venueCapacity": 5000, "ageRestriction": 18}
```

and the request body sends:

```
{"venueCapacity": 8000, "sponsors": ["Pepsi", "Samsung"]}
```

The result should be:

```
{"organizer": "LiveNation", "venueCapacity": 8000, "ageRestriction": 18, "sponsors": ["Pepsi",
    "Samsung"]}
```

Save the updated data and return the updated event record.

Test scenario:

a) Create an event with details `{"organizer":"LiveNation","venueCapacity":5000,"ageRestriction":18}`.

b) `PUT /api/events/{id}/details` with body `{"venueCapacity":8000,"sponsors":["Pepsi","Samsung"]}`.

c) Verify response: organizer=LiveNation (kept), venueCapacity=8000 (updated), ageRestriction=18 (kept), sponsors added.

d) Test with non-existent event → 404.

9.2.3 [S2-F3] Get Event Booking Revenue Summary (DTO)

Branch: `feat/event/S2-F3/<ID>`
Endpoint: `GET /api/events/{id}/revenue?startDate={date}&endDate={date}`
Response DTO (`EventRevenueDTO`): `eventId`, `name`, `totalBookings`, `totalRevenue`, `averageBookingAmount`.
Behavior: Find event –***throws 404 if not found***. Aggregate on `bookings` table for this event in date range and return the required DTO.

Test scenario:

a) Create an event. Create 5 completed bookings for this event in March with amounts 300, 500, 700, 900, 1100.

b) `GET /api/events/{id}/revenue?startDate=2026-03-01&endDate=2026-03-31` → totalBookings=5, totalRevenue=3500, averageBookingAmount=700.

c) Test with date range where no bookings exist → all zeroes.

d) Test with non-existent event → 404.

9.2.4 [S2-F4] Update Event Status (Transactional)

Branch: `feat/event/S2-F4/<ID>`
Endpoint: `PUT /api/events/{id}/status`
Request body: `status`.
Behavior (Transactional): Find event –***throws 404 if not found***. If going CANCELLED, check no

active bookings reference this event —***throws 400 if not***. Update status and save. No need to return it just ***return status code 200***.

Test scenario:

a) Create an event (UPCOMING). Create a CONFIRMED booking for it.

b) `PUT /api/events/{id}/status` with `{"status":"CANCELLED"}` → 400 (active booking exists).

c) Complete the booking. Now try CANCELLED again → 200.

d) Verify event status is CANCELLED.

9.2.5 [S2-F5] Filter Events by Detail Attribute (JSONB)

Branch: `feat/event/S2-F5/<ID>`
Endpoint: `GET /api/events/details/search?key={key}&value={value}&status={status}`
Behavior: Filter events by a JSONB `details` key-value pair and optional status (the status is not required and means if the event is UPCOMING, ONGOING, COMPLETED, or CANCELLED).

Test scenario:

a) Create 3 events: one with details `{"organizer":"LiveNation"}` (UPCOMING), one with `{"organizer":"AEG"}` (UPCOMING), one with `{"organizer":"LiveNation"}` (CANCELLED).

b) `GET /api/events/details/search?key=organizer&value=LiveNation&status=UPCOMING` → returns 1.

c) `GET /api/events/details/search?key=organizer&value=LiveNation` → returns 2 (no status filter).

d) `GET /api/events/details/search?key=organizer&value=Unknown` → empty list.

9.2.6 [S2-F6] Top Rated Events Report (DTO)

Branch: `feat/event/S2-F6/<ID>`
Endpoint: `GET /api/events/reports/top-rated?limit={n}`
Response DTO (`TopEventDTO`): `eventId`, `name`, `rating`, `totalBookings`.
Behavior: Get the top rated (**n**) events and return them as TopEventDTO.

Test scenario:

a) Create 3 events with ratings 4.9, 4.5, 4.2. Create some completed bookings for each.

b) `GET /api/events/reports/top-rated?limit=2` → returns 2, 4.9 first.

c) `GET /api/events/reports/top-rated?limit=10` → returns all 3.

9.2.7 [S2-F7] Rate an Event After Attendance (Transactional)

Branch: `feat/event/S2-F7/<ID>`
Endpoint: `POST /api/events/{id}/rate`
Request body: `bookingId` and `rating` (1–5).
Behavior (Transactional): Find event –***throws 404 if not found***. Verify booking exists and references this event and is COMPLETED —***throws 400 if the booking doesn't reference this event or it's not completed or the rate is not between 1 and 5 and throws 404 if the booking is not found***. Recalculate running average. Update event and Save. Return status ***code 200***.

Test scenario:

a) Create an event (rating=0, totalRatings=0). Create a COMPLETED booking for this event.

b) `POST /api/events/{id}/rate` with `{"bookingId":1,"rating":5}` → 200. Verify rating=5.0, totalRatings=1.

c) Rate again with a different completed booking, rating=3 → verify rating=4.0, totalRatings=2.

d) Test with rating=6 → 400. Test with non-completed booking → 400. Test with booking for another event → 400.

9.2.8 [S2-F8] Verify Event Session (Transactional + Relationship)

Branch: `feat/event/S2-F8/<ID>`
Endpoint: `PUT /api/events/{eventId}/sessions/{sessionId}/verify`
Request body: ***verifiedBy***.
Behavior (Transactional): Find event –***throws 404 if not found***. Find the EventSession by ID and verify it belongs to this event –***throws 404 if session not found*** and –***throws 400 if it doesn't belong to the specified event***. Check the session's `startTime` is in the future —***throws 400 if in the past*** (can't verify a session that already happened). Set `verified = true`. Update the session's JSONB `metadata` to add `verifiedAt` timestamp and `verifiedBy` (from request body). In addition, verify the verifiedBy that is the userID being sent in the request body is an actual ***Admin*** User –(***throw 403 if not***). Return the updated event with all sessions.

Test scenario:

a) Create an event. Add an EventSession with startTime in the future, verified=false. Create an ADMIN user.

b) `PUT /api/events/{eventId}/sessions/{sessionId}/verify` with `{"verifiedBy":3}` → verify session is now verified=true and metadata has verifiedAt and verifiedBy.

c) Test with past startTime → 400. Test with non-ADMIN verifier → 403.

d) Test with session that belongs to different event → 400.

9.2.9 [S2-F9] Get Events with Unverified Sessions (Relationship + Report DTO)

Branch: `feat/event/S2-F9/<ID>`
Endpoint: `GET /api/events/sessions/unverified`
Response DTO (`EventSessionAlertDTO`): `eventId`, `eventName`, `eventStatus`, `unverifiedSessions` (list of sessions where `verified = false`), `unverifiedCount`.
Behavior: Find all events that have at least one unverified session. For each matching event, load their unverified sessions. Build list of DTOs. Only include events that match the criteria.

Test scenario:

a) Create 3 events. Add sessions: Event A has 1 unverified + 2 verified, Event B has all verified, Event C has 3 unverified.

b) `GET /api/events/sessions/unverified` → returns 2 DTOs (Event A with unverifiedCount=1, Event C with unverifiedCount=3). Event B not included.

c) Test when all sessions are verified → empty list.

9.3 Booking Service Features (port 8083)

9.3.1 [S3-F1] Get Bookings by Status and Date Range

Branch: `feat/booking/S3-F1/<ID>`
Endpoint: `GET /api/bookings/search?status={s}&startDate={d}&endDate={d}`

Behavior: Query with date range on `bookingDate` and optional status of the booking. Order by most recent.

Test scenario:

a) Create 5 bookings: 2 COMPLETED in March, 1 PENDING in March, 2 COMPLETED in February.

b) `GET /api/bookings/search?status=COMPLETED&startDate=2026-03-01&endDate=2026-03-31` → returns 2, most recent first.

c) `GET /api/bookings/search?startDate=2026-03-01&endDate=2026-03-31` → returns 3 (no status filter).

9.3.2 [S3-F2] Confirm Booking and Assign Event (Transactional)

Branch: `feat/booking/S3-F2/<ID>`
Endpoint: `PUT /api/bookings/{bookingId}/confirm?eventId={eventId}`
Behavior (Transactional): Find booking –***throws 404 if not found***. Validate booking status is PENDING (–***throws 400 if not valid***—can only confirm a pending booking). Verify the event exists and has status UPCOMING (***throws 404 if not found, and 400 if not upcoming***). Set the booking's `eventId` to the given event. Set booking status to CONFIRMED. Set `confirmedAt` to now. Save booking. Return the updated booking.

Test scenario:

a) Create a PENDING booking (no event). Create an UPCOMING event.

b) `PUT /api/bookings/{bookingId}/confirm?eventId={eventId}` → booking status=CONFIRMED, eventId set, confirmedAt set.

c) Try confirming again → 400 (booking not PENDING anymore).

d) Try confirming with a CANCELLED event → 400.

e) Try with non-existent booking → 404. Non-existent event → 404.

9.3.3 [S3-F3] Get Booking Cost Estimate (DTO)

Branch: `feat/booking/S3-F3/<ID>`
Endpoint: `POST /api/bookings/estimate`
Request body: `eventId`, `ticketCount`, `ticketTier` ("VIP" or "standard").
Response DTO (`BookingCostEstimateDTO`): `ticketCost`, `serviceFee`, `estimatedTotal`, `demandMultiplier`.

Scenario: Before making a booking, an attendee wants to see an estimated total — just like when ticketing sites show "Total: EGP 2,400 (incl. fees)" before checkout. This endpoint computes that estimate. It does **not** create a booking in the database — it only calculates and returns the result.

Behavior — step by step:

a) **Calculate ticket cost:** Use the average session capacity from the event's `event_sessions` table as a proxy for base price: `basePrice = AVG(capacity) / 10`. Apply a tier multiplier: VIP = 2.5, standard = 1.0. Then:

```
ticketCost = basePrice * tierMultiplier * ticketCount
```

For example, if average session capacity is 500, base price is 50, tier is VIP (2.5x), count is 2: `50 * 2.5 * 2 = 250 EGP`.

b) **Calculate service fee:** Fixed at 15% of ticket cost:

```
serviceFee = ticketCost * 0.15
```

For the example: `250 * 0.15 = 37.5 EGP`.

c) **Determine demand multiplier:** Count how many active bookings (status IN `PENDING`, `CONFIRMED`, `CHECKED_IN`) reference the same event.

Based on the count, apply the following demand tiers:

- 0–50 active bookings → `demandMultiplier = 1.0` (normal)
- 51–200 active bookings → `demandMultiplier = 1.25` (popular)
- More than 200 active bookings → `demandMultiplier = 1.5` (high demand)

d) **Calculate total:**

```
estimatedTotal = (ticketCost + serviceFee) * demandMultiplier
```

For the example with no demand surge: `(250 + 37.5) * 1.0 = 287.5 EGP`.
Same during high demand: `(250 + 37.5) * 1.5 = 431.25 EGP`.

e) **Build and return the DTO** containing all four values.

Test scenario:

a) Create an event with sessions averaging capacity 500.

b) `POST /api/bookings/estimate` with eventId=1, ticketCount=2, ticketTier="VIP".

c) Verify ticketCost=250, serviceFee=37.5, demandMultiplier=1.0, estimatedTotal=287.5.

d) Create 60 active bookings for the same event. Call estimate again → demandMultiplier should be 1.25.

e) Verify no booking was created in the database (this endpoint is read-only).

9.3.4 [S3-F4] Complete Booking (Transactional Multi-Step)

Branch: `feat/booking/S3-F4/<ID>`
Endpoint: `PUT /api/bookings/{id}/complete`
Behavior (Transactional): Find booking –***throws 404 if not found***. Validate status CHECKED_IN –***throws 400 if not***. Set COMPLETED. Calculate totalAmount if unset (sum of BookingItem quantities * unitPrices). Create ticket sale record with status PENDING. Save booking. Return the booking after the update.

Test scenario:

a) Create a booking with status CHECKED_IN and 3 BookingItems (quantities 2, 1, 4 at prices 100, 250, 50).

b) `PUT /api/bookings/{id}/complete` → booking status=COMPLETED, totalAmount=650, a new PENDING ticket sale created.

c) Try completing again → 400 (already COMPLETED).

d) Try completing a PENDING booking → 400.

9.3.5 [S3-F5] Filter Bookings by Metadata Field (JSONB)

Branch: `feat/booking/S3-F5/<ID>`
Endpoint: `GET /api/bookings/metadata/search?key={key}&value={value}`
Behavior: Validate key/value –***throws 400 if not valid***. Return matches.

Test scenario:

a) Create 3 bookings: one with metadata `{"ticketTier":"VIP"}`, two with `{"ticketTier":"standard"}`.

b) `GET /api/bookings/metadata/search?key=ticketTier&value=VIP` → returns 1 booking.

c) `GET /api/bookings/metadata/search?key=&value=x` → 400.

9.3.6 [S3-F6] Booking Analytics by Time Period (Report DTO)

Branch: `feat/booking/S3-F6/<ID>`
Endpoint: `GET /api/bookings/analytics?startDate={d}&endDate={d}`
Response DTO (`BookingAnalyticsDTO`)**:** `totalBookings`, `completedBookings`, `cancelledBookings`, `totalRevenue`, `averageBookingAmount`, `completionRate`.
Behavior: Calculate the required fields for bookings from the start date till the end date. Build the DTO and return it.

Test scenario:

a) Create 10 bookings in March: 7 COMPLETED (amounts: 200, 300, 400, 500, 600, 700, 800), 3 CANCELLED.

b) `GET /api/bookings/analytics?startDate=2026-03-01&endDate=2026-03-31` → totalBookings=10, completedBookings=7, cancelledBookings=3, totalRevenue=3500, averageBookingAmount=500, completionRate=70.0.

9.3.7 [S3-F7] Cancel Booking (Transactional)

Branch: `feat/booking/S3-F7/<ID>`
Endpoint: `PUT /api/bookings/{id}/cancel`
Behavior (Transactional): Find booking –***throws 404 if not found***. Validate status PENDING/CONFIRMED –***throws 400 if not valid***. Set CANCELLED. Cancel all VALID tickets for this booking. Save and ***return status code 200*** if the workflow was done successfully.

Test scenario:

a) Create a CONFIRMED booking with an event assigned and 3 VALID tickets.

b) `PUT /api/bookings/{id}/cancel` → booking status=CANCELLED, all tickets set to CANCELLED.

c) Try cancelling a COMPLETED booking → 400.

d) Try cancelling a CHECKED_IN booking → 400.

9.3.8 [S3-F8] Add Items to Existing Booking (Transactional + Relationship)

Branch: `feat/booking/S3-F8/<ID>`
Endpoint: `POST /api/bookings/{bookingId}/items`
Request body: A list of item objects, each with `sessionId`, `sessionTitle`, `quantity`, `unitPrice`, and `metadata`.
Behavior (Transactional): Find booking –***throws 404 if not found***. Validate booking status is PENDING or CONFIRMED (–***throws 400 if not valid***—cannot add items to checked-in or completed bookings). Validate each item has sessionId, sessionTitle, quantity, and unitPrice –***throws 400 if not valid***. Determine the `eventOrder` for each new item by continuing from the booking's current max event

order (or starting at 1 if no items exist). Create `BookingItem` entities with status = RESERVED and add them to the booking. Save the booking. Return the updated booking with all its items, ordered by `eventOrder`.

Test scenario:

a) Create a PENDING booking with no items.

b) `POST /api/bookings/{bookingId}/items` with 2 items → verify items created with eventOrder 1 and 2, status=RESERVED.

c) Add 1 more item → verify eventOrder=3.

d) Try adding items to a COMPLETED booking → 400.

e) Try adding an item with missing sessionTitle → 400.

9.3.9 [S3-F9] Get Booking Details with Items (Relationship DTO)

Branch: `feat/booking/S3-F9/<ID>`
Endpoint: `GET /api/bookings/{bookingId}/details`
Response DTO (`BookingDetailsDTO`): `bookingId`, `userId`, `eventId`, `status`, `totalAmount`, `metadata` (JSONB), `items` (list of item objects ordered by `eventOrder`, each with id, eventOrder, sessionTitle, quantity, unitPrice, status, metadata), `totalItems`, `confirmedItems` (count where status = CONFIRMED).
Behavior: Find booking with its items loaded (–***throws 404 if not found***). Count items by status (Confirmed and not Confirmed). Build the DTO with the full item list ordered by `eventOrder` ascending.

Test scenario:

a) Create a booking with 4 items: 2 CONFIRMED, 2 RESERVED.

b) `GET /api/bookings/{bookingId}/details` → verify DTO has totalItems=4, confirmedItems=2, items ordered by eventOrder.

c) Test with booking that has no items → totalItems=0, confirmedItems=0, empty items list.

d) Test with non-existent booking → 404.

9.4 Ticket Service Features (port 8084)

9.4.1 [S4-F1] Get Latest Ticket for a Booking

Branch: `feat/ticket/S4-F1/<ID>`
Endpoint: `GET /api/tickets/booking/{bookingId}/latest`
Behavior: Verify booking exists (–***throws 404 if not found***). Query most recent ***ticket*** by `issuedAt`. ***throws 404 if none***.

Test scenario:

a) Create a booking. Issue 3 tickets with different timestamps.

b) `GET /api/tickets/booking/{bookingId}/latest` → should return the most recent ticket.

c) Test with booking that has no tickets → 404.

d) Test with non-existent booking → 404.

9.4.2 [S4-F2] Issue Ticket with Metadata (JSONB)

Branch: `feat/ticket/S4-F2/<ID>`
Endpoint: `POST /api/tickets/booking/{bookingId}`
Request body: attendeeName, ticketCode, metadata map.
Behavior: Verify booking exists –***throws 404 if not found***. Create Ticket with issuedAt = now, status = VALID. Save. Return ***status code 201***.

Test scenario:

a) Create a booking.

b) `POST /api/tickets/booking/{bookingId}` with attendeeName="Ahmed", ticketCode="TIX-2026-001", metadata=`{"seatNumber":"A12","section":"VIP","gate":"Gate3"}` → 201.

c) Verify the saved ticket has the correct bookingId and metadata.

d) Test with non-existent booking → 404.

9.4.3 [S4-F3] Find Tickets Near an Event Venue (DTO with Distance)

Branch: `feat/ticket/S4-F3/<ID>`
Endpoint: `GET /api/tickets/nearby?lat={lat}&lon={lon}&radiusKm={r}`
Response DTO (`NearbyTicketDTO`): `ticketId`, `attendeeName`, `bookingId`, `eventName`, `eventLat`, `eventLon`, `distanceKm`.
Behavior: Join tickets with bookings and events. Use event venue coordinates from the `events.details` JSONB fields `venueLat` and `venueLon`. Calculate distance (euclidean distance[2] * 111), filter VALID tickets by radius, sort ascending. Map to DTOs.

Test scenario:

a) Create events at different locations with JSONB venueLat/venueLon. Create bookings and VALID tickets.

b) `GET /api/tickets/nearby?lat=30.04&lon=31.23&radiusKm=5` → returns tickets for nearby events sorted by distance.

c) Test with radiusKm=0.01 → might return empty or only closest.

9.4.4 [S4-F4] Batch Ticket Issuance (Transactional)

Branch: `feat/ticket/S4-F4/<ID>`
Endpoint: `POST /api/tickets/batch`
Request body: `bookingId` and list of ticket objects (each with attendeeName, ticketCode, metadata).
Behavior (Transactional): Verify booking –***throws 404 if not found***. Validate ticket codes are unique –***throws 400 if duplicates found***. Save all tickets with issuedAt = now, status = VALID. Return ***status code 201 with count***.

Test scenario:

a) Create a booking.

b) `POST /api/tickets/batch` with bookingId and 3 ticket objects → 201 with count=3.

c) Verify all 3 tickets saved with correct bookingId.

d) Test with duplicate ticketCode in the batch → 400.

e) Test with non-existent booking → 404.

[2] euclidean distance = sqrt($(x2 - x1)^2 + (y2 - y1)^2$)

9.4.5 [S4-F5] Filter Tickets by Metadata (JSONB Query)

Branch: `feat/ticket/S4-F5/<ID>`
Endpoint: `GET /api/tickets/metadata/search?key={k}&operator={op}&value={v}`
Operators: `eq`, `gt`, `lt`.
Behavior: Validate operator –***throws 400 if not valid***. For gt/lt cast JSONB to numeric. Return matches.

Test scenario:

a) Create tickets with metadata: gate=1, gate=3, gate=5.

b) `GET /api/tickets/metadata/search?key=gate&operator=gt&value=2` → returns 2 tickets (gate 3 and 5).

c) `GET /api/tickets/metadata/search?key=gate&operator=eq&value=1` → returns 1.

d) `GET /api/tickets/metadata/search?key=gate&operator=xyz&value=1` → 400.

9.4.6 [S4-F6] Get Tickets in Date Range

Branch: `feat/ticket/S4-F6/<ID>`
Endpoint: `GET /api/tickets/history?startDate={d}&endDate={d}&status={s}`
Behavior: Query all tickets within the date range on `issuedAt` with optional status filter. Order by issuedAt ascending.

Test scenario:

a) Create 5 bookings. Issue a ticket for each: 3 with issuedAt in March (2 VALID, 1 USED), 2 with issuedAt in February.

b) `GET /api/tickets/history?startDate=2026-03-01&endDate=2026-03-31` → returns 3 tickets ordered by issuedAt ascending.

c) `GET /api/tickets/history?startDate=2026-03-01&endDate=2026-03-31&status=VALID` → returns 2.

d) Test with date range where no tickets exist → empty list.

9.4.7 [S4-F7] Purge Expired Tickets (Transactional)

Branch: `feat/ticket/S4-F7/<ID>`
Endpoint: `DELETE /api/tickets/purge?olderThanDays={n}`
Behavior (Transactional): Calculate cutoff (if the request says olderThanDays=30, then cutoff = today minus 30 days). Count then delete via native `@Modifying` query (only EXPIRED and CANCELLED tickets). Return deleted count.

Test scenario:

a) Create 10 tickets: 7 with status EXPIRED and issuedAt 60 days ago, 3 with status VALID and issuedAt 60 days ago.

b) `DELETE /api/tickets/purge?olderThanDays=30` → returns deletedCount=7 (only EXPIRED/CANCELLED).

c) Verify the 3 VALID tickets remain.

9.4.8 [S4-F8] Event Attendance Summary (DTO)

Branch: `feat/ticket/S4-F8/<ID>`
Endpoint: `GET /api/tickets/event/{eventId}/summary`
Response DTO (`EventAttendanceSummaryDTO`): `eventId`, `totalTickets`, `usedTickets`, `validTickets`, `attendanceRate`, `lastCheckIn`.
Behavior: Join tickets with bookings to find all tickets for this event. Calculate the metrics: usedTickets = count where status=USED, validTickets = count where status=VALID, attendanceRate = usedTickets/totalTickets*100. ***Throw 404 if no tickets found for this event.***

Test scenario:

a) Create an event. Create bookings for it. Issue 10 tickets: 6 USED, 3 VALID, 1 EXPIRED.

b) `GET /api/tickets/event/{eventId}/summary` → totalTickets=10, usedTickets=6, validTickets=3, attendanceRate=60.0.

c) Test with event that has no tickets → 404.

9.4.9 [S4-F9] Find Unused Tickets for Upcoming Events (JSONB + Cross-Table DTO)

Branch: `feat/ticket/S4-F9/<ID>`
Endpoint: `GET /api/tickets/unused-upcoming`
Response DTO (`UnusedTicketDTO`): `ticketId`, `attendeeName`, `ticketCode`, `bookingId`, `eventName`, `eventDate`.
Behavior: Find VALID tickets whose associated booking references an UPCOMING event (join through bookings to events). Extract event details. Map results to DTOs.

Test scenario:

a) Create 2 events: one UPCOMING, one COMPLETED. Create bookings and VALID tickets for both.

b) `GET /api/tickets/unused-upcoming` → returns only tickets for the UPCOMING event.

c) Test when all events are COMPLETED → empty list.

9.5 Sales Service Features (port 8085)

9.5.1 [S5-F1] Get Ticket Sales by Status and Date Range

Branch: `feat/sales/S5-F1/<ID>`
Endpoint: `GET /api/sales/search?status={s}&startDate={d}&endDate={d}`
Behavior: Query with optional status and date range on `createdAt`. Return results with most recent first.

Test scenario:

a) Create 5 ticket sales: 2 COMPLETED in March, 1 REFUNDED in March, 2 COMPLETED in February.

b) `GET /api/sales/search?status=COMPLETED&startDate=2026-03-01&endDate=2026-03-31` → returns 2, most recent first.

c) `GET /api/sales/search?startDate=2026-03-01&endDate=2026-03-31` → returns 3 (no status filter).

9.5.2 [S5-F2] Process Refund (Transactional + JSONB Update)

Branch: `feat/sales/S5-F2/<ID>`
Endpoint: `PUT /api/sales/{id}/refund`
Request body: `reason` (string).
Behavior (Transactional): Find ticket sale –***throws 404 if not found***. Validate status COMPLETED –***throws 400 if not***. Set REFUNDED. Add refundReason and refundedAt to JSONB transactionDetails. Save and return the updated ticket sale.

Test scenario:

a) Create a COMPLETED ticket sale.

b) `PUT /api/sales/{id}/refund` with `{"reason":"event cancelled"}` → status=REFUNDED, transactionDetails has refundReason and refundedAt.

c) Try refunding again → 400. Try refunding a FAILED ticket sale → 400.

9.5.3 [S5-F3] User Ticket Sale Summary (DTO)

Branch: `feat/sales/S5-F3/<ID>`
Endpoint: `GET /api/sales/user/{userId}/summary`
Response DTO (`UserSaleSummaryDTO`)**:** `userId`, `totalSales`, `totalAmount`, `methodBreakdown` (a `Map<String, Double>`).

Behavior — step by step:

a) **Verify user exists:** Check if the user exists –***throws 404 if not found***.

b) **Query ticket sale data grouped by method:** Query the ticket sales of the user that have status COMPLETED and then group the results by the method of payment to compute the count and the sum of `amount` per each method.

c) **Build the method breakdown map:** Populate a `Map<String, Double>` where each key is the payment method (e.g., `"CREDIT_CARD"`) and the value is the total amount paid via that method.

d) **Calculate totals:** `totalSales` is the sum of all counts. `totalAmount` is the sum of all amounts.

e) **Build and return the DTO**.

Test scenario:

a) Create a user. Create 4 COMPLETED ticket sales: 2 via CREDIT_CARD (300, 500), 1 via DEBIT_CARD (200), 1 via WALLET (150).

b) `GET /api/sales/user/{userId}/summary` → totalSales=4, totalAmount=1150, methodBreakdown=`{"CREDIT_CARD":800,"DEBIT_CARD":200,"WALLET":150}`.

c) Test with non-existent user → 404.

9.5.4 [S5-F4] Process Ticket Sale for Booking (Transactional)

Branch: `feat/sales/S5-F4/<ID>`
Endpoint: `POST /api/sales/booking/{bookingId}`
Request body: `method`, `cardLastFour` (optional).
Behavior (Transactional): Verify booking exists –***throws 404 if not found*** and is COMPLETED –***throws 400 if having different status***. Check no COMPLETED ticket sale exists for this booking –***throws 400 if not with a message "already paid"***. Update the TicketSale that was created upon booking completion with status PENDING, populate JSONB. Save. Return ***status code 201*** indicating successful payment.

Test scenario:

a) Create a COMPLETED booking.

b) `POST /api/sales/booking/{bookingId}` with `{"method":"CREDIT_CARD","cardLastFour":"4242"}` → 201. Verify ticket sale updated with correct amount and JSONB populated.

c) Try paying again for the same booking → 400 ("already paid"). Try paying for a PENDING booking → 400.

9.5.5 [S5-F5] Apply Promotion to Ticket Sale (Transactional + Join Entity)

Branch: `feat/sales/S5-F5/<ID>`
Endpoint: `POST /api/sales/{saleId}/promotions/{promotionId}`
No request body — both IDs come from the URL path.

Scenario: A customer has a pending ticket sale of 800 EGP for concert tickets and wants to apply a promo code before checkout. The system validates the promotion, calculates the discount, and creates a `SalePromotion` join entity linking the two — the same pattern as `OrderItem` from Lab 4.

Behavior (Transactional) — step by step:

a) **Find ticket sale** by `saleId` — *throw 404 if not found.*

b) **Validate ticket sale status:** The ticket sale's `status` must be `PENDING` — *throw 400 with message "cannot apply promotion to a completed/cancelled sale" if status is `COMPLETED` or `REFUNDED`.*

c) **Find promotion** by `promotionId` — *throw 404 if not found.*

d) **Validate promotion is usable:**

- Promotion's `active` field must be `true` — *throw 400 if not.*
- Promotion's `expiryDate` must be in the future — *throw 400 if expired.*
- Promotion's `currentUses` must be less than `maxUses` — *throw 400 if limit reached.*

e) **Check for duplicate:** Query `SalePromotion` records — *throw 400 with message "promotion already applied" if found.*

f) **Calculate the discount amount** based on the promotion's `discountType`:

- If `PERCENTAGE`: `discount = sale.amount * promotion.discountValue / 100`
  Example: 800 EGP sale with a 25% promotion → `800 * 25 / 100 = 200 EGP` discount.
- If `FIXED`: `discount = promotion.discountValue`
  Example: 800 EGP sale with a 100 EGP fixed promotion → 100 EGP discount.

Then **cap the discount**: if the calculated discount exceeds the sale amount, set it to the sale amount (a promotion cannot produce a negative total).

g) **Create the join entity:** Create a new `SalePromotion` with the computed variables.

h) **Update the promotion:** Increment `currentUses` by 1.

i) **Save** the `SalePromotion` and the updated promotion and ticket sale.

j) **Return** the updated ticket sale with its list of applied promotions.

Test scenario:

a) Create a PENDING ticket sale (amount=800). Create a promotion: code="SHOW25", PERCENTAGE, discountValue=25, maxUses=3.

b) `POST /api/sales/{saleId}/promotions/{promotionId}` → SalePromotion created with discountApplied=200. Promotion currentUses incremented to 1.

c) Apply same promotion again → 400. Create a FIXED promotion with discountValue=9999, apply → discountApplied capped at 800.

d) Test with expired promotion → 400. Test with inactive promotion → 400.

e) Test applying promotion to a COMPLETED ticket sale → 400.

9.5.6 [S5-F6] Revenue Report by Date Range (Report DTO)

Branch: `feat/sales/S5-F6/<ID>`
Endpoint: `GET /api/sales/reports/revenue?startDate={d}&endDate={d}`
Response DTO (`RevenueReportDTO`): `totalRevenue`, `totalTransactions`, `averageSale`, `refundedAmount`, `refundCount`.

Scenario: An admin wants to see how much money the platform made in a given period, and how much was lost to refunds.

Behavior — step by step:

a) **Validate dates:** ***Throw 400 if*** *`startDate`* ***is after*** *`endDate`*.

b) **Query ticket sales in date range.**

c) **The five metrics:**

- `totalRevenue` — Sum of `amount` where `status = 'COMPLETED'`.
- `totalTransactions` — Count where `status = 'COMPLETED'`.
- `averageSale` — `totalRevenue / totalTransactions`. Handle division by zero.
- `refundedAmount` — Sum of `amount` where `status = 'REFUNDED'`.
- `refundCount` — Count where `status = 'REFUNDED'`.

d) **Build and return the DTO.**

Test scenario:

a) Create ticket sales in March: 5 COMPLETED (amounts: 200, 400, 600, 800, 1000 = total 3000), 2 REFUNDED (amounts: 300, 500 = total 800).

b) `GET /api/sales/reports/revenue?startDate=2026-03-01&endDate=2026-03-31` → totalRevenue=3000, totalTransactions=5, averageSale=600, refundedAmount=800, refundCount=2.

c) Test with startDate after endDate → 400.

9.5.7 [S5-F7] Retry Failed Ticket Sale (Transactional)

Branch: `feat/sales/S5-F7/<ID>`
Endpoint: `PUT /api/sales/{id}/retry`
No request body — the ticket sale ID comes from the URL path.

Scenario: A ticket sale failed (e.g., card declined). The customer wants to try again. Instead of creating a new sale, the system retries the existing one.

Behavior (Transactional) — step by step:

a) **Find ticket sale** by `id` — *throw 404 if not found.*

b) **Validate status:** Must be `FAILED` — *throw 400 if anything else.*

c) **Update status:** Set to `COMPLETED`.

d) **Update JSONB `transactionDetails`:**

- `retryAttempt` — increment by 1.
- `gatewayResponse` — overwrite with `"approved"`.

So if the JSONB was:

```
{
  "gatewayResponse":"declined",
  "cardLastFour":"4242",
  "retryAttempt":0,
  "failureReason":"card expired"
}
```

After retry it becomes:

```
{
  "gatewayResponse":"approved",
  "cardLastFour":"4242",
  "retryAttempt":1,
  "failureReason":"card expired"
}
```

e) **Save** the ticket sale.

f) **Return** the updated ticket sale entity.

Test scenario:

a) Create a FAILED ticket sale with transactionDetails `{"gatewayResponse":"declined","retryAttempt":0}`.

b) `PUT /api/sales/{id}/retry` → status=COMPLETED, retryAttempt=1, gatewayResponse="approved".

c) Try retrying a COMPLETED ticket sale → 400. Try retrying a REFUNDED ticket sale → 400.

9.5.8 [S5-F8] Get Ticket Sale Details with Applied Promotions (Join Entity DTO)

Branch: `feat/sales/S5-F8/<ID>`
Endpoint: `GET /api/sales/{saleId}/details`
Response DTO (`SaleDetailsDTO`): `saleId`, `bookingId`, `userId`, `originalAmount`, `method`, `status`, `transactionDetails` (JSONB), `appliedPromotions` (list of objects, each with `promotionCode`, `discountType`, `discountApplied`, `appliedAt`), `totalDiscount` (sum of all discounts), `finalAmount` (`originalAmount - totalDiscount`).
Behavior: Find ticket sale with its `SalePromotion` entries loaded (–***throws 404 if not found***). For each `SalePromotion`, access the linked Promotion entity to get the promotion code and discount type. Calculate totalDiscount and finalAmount. Build the DTO and return it.

Test scenario:

a) Create a ticket sale (amount=800). Apply 2 promotions: discountApplied=200 and discountApplied=100.

b) `GET /api/sales/{saleId}/details` → originalAmount=800, appliedPromotions has 2 entries, totalDiscount=300, finalAmount=500.

c) Test with ticket sale that has no promotions → totalDiscount=0, finalAmount=originalAmount.

d) Test with non-existent ticket sale → 404.

9.5.9 [S5-F9] Get Most Used Promotions Report (Join Entity + Aggregation)

Branch: `feat/sales/S5-F9/<ID>`
Endpoint: `GET /api/sales/promotions/top-used?limit={n}`
Response DTO (`PromotionUsageDTO`): `promotionId`, `code`, `discountType`, `discountValue`, `timesUsed`, `totalDiscountGiven`, `active`, `expired`.

Scenario: A platform manager wants to see which promotions are performing best.

Behavior — step by step:

a) **Write a query** that searches for `promotions` with `sale_promotions`. Group by promotion. For each promotion, compute:

- `timesUsed` — the promotion's `currentUses` value.
- `totalDiscountGiven` — `SUM(sale_promotions.discount_applied)`. This is the total money saved by attendees who used this promotion.

For example, if promotion "SHOW25" was applied to 3 sales with discounts of 200, 200, and 150 EGP, then `timesUsed = 3` and `totalDiscountGiven = 550.0`.

b) **Order by** `timesUsed` descending and **limit** to `n` results.

c) **Map each result to a `PromotionUsageDTO`:** Populate all fields. `expired` is a computed boolean in Java.

d) **Return** the list of DTOs.

Test scenario:

a) Create 3 promotions. Apply promotion A to 5 sales (total discount=1000), promotion B to 3 sales (total=450), promotion C to 1 sale (total=100).

b) `GET /api/sales/promotions/top-used?limit=2` → returns promotion A (timesUsed=5, totalDiscountGiven=1000) then promotion B.

c) Verify "expired" field is true for promotions past their expiryDate.

10 Git Workflow — Feature Development

Every team member follows this workflow for **every feature**:

a) Start from main: `git checkout main && git pull origin main`

b) Create feature branch: `git checkout -b feat/<service>/<feature-ID>/<YOUR-STUDENT-ID>`

c) Implement: repository methods → service logic → controller endpoint → test via Postman.

d) Commit: `git commit -m "feat(<service-name>): <description> (<YOUR-STUDENT-ID>)"` (except only the first commit which is the initialization of the project)

e) Push and create a Pull Request on GitHub.

f) At least 1 teammate reviews and approves.

g) **Merge into main using a regular merge commit ("Create a merge commit" on GitHub). Do NOT use squash merge.** A regular merge preserves the branch name in the merge commit message (e.g., `Merge pull request #12 from feat/user/S1-F1/55-8078`), which the auto-grader uses to verify who implemented which feature.

h) **Do NOT delete the feature branch after merging.** The auto-grader will verify that each feature has a correctly-named branch containing the student ID. If you delete the branch, the grader will not be able to verify it and this may result in deductions in your grade.

11 Dockerization (Phase D)

a) Create a `Dockerfile` inside each service module using `eclipse-temurin:25.0.2_10-jdk`. Copy that service's JAR and expose port 8080.

b) Update the root `docker-compose.yaml` to include all 5 services alongside PostgreSQL. Each service:

- Builds from its own Dockerfile (e.g., `build: ./user-service`)
- Maps its host port to container port 8080 (e.g., `8081:8080`)
- Overrides datasource URL to `jdbc:postgresql://postgres:5432/eventticketingdb`
- Depends on the PostgreSQL service

c) Build: `mvn clean package -DskipTests` from the root.

d) Run: `docker compose up --build`

e) Verify all services respond:

- `localhost:8081/api/users` (User Service)
- `localhost:8082/api/events` (Event Service)
- `localhost:8083/api/bookings` (Booking Service)
- `localhost:8084/api/tickets` (Ticket Service)
- `localhost:8085/api/sales` (Sales Service)

f) Git: branch `feat/docker/<ID>`, commit with ID, PR, merge.

12 Work Distribution (Recommended):

Each member implements around 3 features. Phase A (all entities + CRUD for every entity) is done collectively (You can divide it across the sub_team that will implement the service or divide it across all the members equally as you wish).

13 Submission & Auto-Grader

a) Repository must be **private**.

b) `Scalable-Submissions` must be added as a collaborator.

c) `team.json` must be present and valid at the project root.

d) `docker compose up` must start PostgreSQL + all 5 services.

e) Services must respond at `localhost:8081` through `localhost:8085`.

f) Git history must show feature branches merged via PRs with student IDs.

g) Commits must follow `feat(<service>): <desc> (<ID>)` format.

h) Details about the usage of the auto grader and the testing will be sent later.

Important: Grading is **per member**. A feature only counts for the member whose GitHub username authored the commits on a branch containing their student ID. Features with mismatched or missing IDs receive zero credit. (The zero will affect both, the team member and the whole team). The only case that the team member will receive a different grade from the rest of the team is if there is no history of commits that shows that this member worked in the milestone (if his/her work was done then the whole team will receive their grade fully and he/she will receive zero. If his/her work was missing from the

overall work, the whole team will be deducted for the missing work and he/she will still receive zero). It is the responsibility of the scrum master to make sure that everyone is working and the whole requirements are being delivered, if there is an issue with any member(s) then a form will be sent for complains that you can fill it.

Deadline: The Milestone deadline is Saturday 11/04/2026 at 11:59 PM

Submission: Submission form will be sent later to the scrum masters.

14 Appendix: JSONB Sample Data

Below is one example per service showing what the JSONB columns look like in practice. A full database seeder will be provided separately.

User.preferences:

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false},
  "favoriteCategory": "CONCERT",
  "ticketTier": "VIP",
  "timezone": "UTC+2"
}
```

Event.details:

```
{
  "organizer": "LiveNation Egypt",
  "venueCapacity": 5000,
  "ticketPriceRange": "200-1500 EGP",
  "ageRestriction": 16,
  "description": "International music festival",
  "sponsors": ["Pepsi", "Samsung"],
  "venueLat": 30.0444,
  "venueLon": 31.2357
}
```

Booking.metadata:

```
{
  "ticketTier": "VIP",
  "specialRequests": "wheelchair access",
  "groupSize": 4,
  "referralCode": "FRIEND20",
  "notes": "Birthday celebration"
}
```

Ticket.metadata:

```
{
  "seatNumber": "A12",
  "section": "VIP",
  "tier": "VIP",
  "gate": "Gate 3",
  "qrCodeUrl": "https://tickets.example.com/qr/TIX-2026-001",
  "checkInTime": null
}
```

TicketSale.transactionDetails:

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "receiptUrl": "https://receipts.example.com/abc",
  "failureReason": null,
  "bookingReference": "BK-2026-0042"
}
```

15 Appendix: Example Database Tables

The following tables show what your database should look like after seeding some test data. Use these as a reference when testing your CRUD operations and features via Postman. All tables coexist in the same `eventticketingdb` database.

Note: JSONB columns are shown as formatted JSON code blocks below each table for readability. In the actual database, each block is stored as a single JSONB column value in the corresponding row.

15.1 User Service Tables

15.1.1 `users` table

id	name	email	password	phone	role	status	created_at
1	Ahmed Hassan	ahmed@mail.com	pass123	+201011111111	ATTENDEE	ACTIVE	2026-03-01 10:00
2	Sara Mohamed	sara@mail.com	pass456	+201022222222	ATTENDEE	ACTIVE	2026-03-02 14:30
3	Admin User	admin@events.com	admin789	+201033333333	ADMIN	ACTIVE	2026-03-01 08:00

JSONB column — `preferences`:

`User 1 (Ahmed):`

```
{
  "language": "ar",
  "notifications": {"email": true, "sms": false},
  "favoriteCategory": "CONCERT",
  "ticketTier": "VIP",
  "timezone": "UTC+2"
}
```

`User 2 (Sara):`

```
{
  "language": "en",
  "notifications": {"email": true, "sms": true},
  "favoriteCategory": "SPORTS",
  "ticketTier": "standard",
  "timezone": "UTC+2"
}
```

`User 3 (Admin):`

```
{
  "language": "en",
  "notifications": {"email": true, "sms": false}
}
```

15.1.2 favorite_venues table

id	user_id	label	venue_name	location	capacity	is_default
1	1	My Go-To	Cairo Opera House	Zamalek, Cairo	1200	true
2	1	Weekend Spot	Cairo Stadium	Nasr City, Cairo	75000	false
3	2	Home Venue	Cairo Festival City Arena	New Cairo	4500	true

JSONB column — metadata:

Venue 1 (Ahmed -- Cairo Opera House):

```
{
  "parkingAvailability": "underground parking, 200 spots",
  "accessibility": {"wheelchair": true, "elevators": true},
  "foodOptions": ["cafe", "fine dining"],
  "websiteUrl": "https://cairoopera.org",
  "nearbyTransport": "Opera metro station (2 min walk)",
  "personalNotes": "Best seats are row C in the main hall"
}
```

Venue 2 (Ahmed -- Cairo Stadium):

```
{
  "parkingAvailability": "open lot, limited on match days",
  "accessibility": {"wheelchair": true, "elevators": false},
  "foodOptions": ["street vendors", "kiosks"],
  "nearbyTransport": "Stadium metro station",
  "personalNotes": "East stand has the best atmosphere"
}
```

Venue 3 (Sara -- Cairo Festival City Arena):

```
{
  "parkingAvailability": "mall parking, free",
  "accessibility": {"wheelchair": true, "elevators": true},
  "foodOptions": ["mall food court", "Starbucks"],
  "websiteUrl": "https://cfcarena.com",
  "nearbyTransport": "Ring Road exit 12, taxi recommended"
}
```

15.2 Event Service Tables

15.2.1 events table

id	name	venue	category	status	rating	total_ratings
1	Cairo Jazz Festival	Cairo Opera House	CONCERT	UPCOMING	4.8	3
2	Al Ahly vs Zamalek	Cairo Stadium	SPORTS	UPCOMING	4.5	2
3	TEDx Cairo	AUC New Campus	CONFERENCE	COMPLETED	4.2	1

(eventDate: Event 1 = 2026-04-15, Event 2 = 2026-05-20, Event 3 = 2026-03-01. createdAt: all = 2026-03-01)

JSONB column — details:

Event 1 (Cairo Jazz Festival):

```
{
  "organizer": "Cairo Jazz Club",
  "venueCapacity": 3000,
```

```
  "ticketPriceRange": "300-1500 EGP",
  "ageRestriction": 18,
  "description": "Annual jazz and world music festival",
  "sponsors": ["Pepsi", "Samsung"],
  "venueLat": 30.0459,
  "venueLon": 31.2243
}
```

Event 2 (Al Ahly vs Zamalek):

```
{
  "organizer": "Egyptian FA",
  "venueCapacity": 75000,
  "ticketPriceRange": "100-500 EGP",
  "ageRestriction": null,
  "description": "Egyptian Premier League derby",
  "sponsors": ["WE", "Etisalat"],
  "venueLat": 30.0694,
  "venueLon": 31.3130
}
```

Event 3 (TEDx Cairo):

```
{
  "organizer": "TEDx Committee",
  "venueCapacity": 800,
  "ticketPriceRange": "500-2000 EGP",
  "ageRestriction": 16,
  "description": "Ideas worth spreading",
  "sponsors": ["Google", "Microsoft"],
  "venueLat": 30.0198,
  "venueLon": 31.5001
}
```

15.2.2 `event_sessions` table

id	event_id	title	speaker	capacity	verified
1	1	Opening Night	Dina El Wedidi	3000	true
2	1	Day 2 — Jazz Fusion	Cairokee	3000	true
3	1	Closing Night	*TBA*	3000	false
4	2	Match Day	–	75000	true
5	3	Morning Talks	Dr. Magdi Yacoub	400	true
6	3	Afternoon Workshop	–	400	false

(startTime/endTime and createdAt omitted — see JSONB metadata for timing details)

JSONB column — `metadata`:

Session 1 (Opening Night):

```
{
  "roomName": "Main Hall",
  "equipment": ["PA system", "stage lighting", "projector"],
  "language": "Arabic",
  "accessibility": {"wheelchair": true, "signLanguage": false},
  "recordingAvailable": true
}
```

Session 5 (Morning Talks):

```
{
  "roomName": "Auditorium A",
  "equipment": ["projector", "microphone", "live stream"],
  "language": "English",
  "accessibility": {"wheelchair": true, "signLanguage": true},
  "recordingAvailable": true
}
```

15.3 Booking Service Tables

15.3.1 `bookings` table

id	user_id	event_id	status	total_amount	booking_date	confirmed_at
1	1	1	COMPLETED	3000.00	2026-03-10 09:00	2026-03-10 09:05
2	2	2	CONFIRMED	600.00	2026-03-18 15:00	2026-03-18 15:10
3	1	3	CANCELLED	1000.00	2026-03-05 11:00	*null*

JSONB column — `metadata`:

`Booking 1 (Ahmed -- Jazz Festival, Completed):`

```
{
  "ticketTier": "VIP",
  "specialRequests": "front row seating",
  "groupSize": 2,
  "referralCode": "FRIEND20",
  "notes": "Anniversary celebration"
}
```

`Booking 2 (Sara -- Football Match, Confirmed):`

```
{
  "ticketTier": "standard",
  "specialRequests": "wheelchair access",
  "groupSize": 3,
  "notes": "Family outing"
}
```

`Booking 3 (Ahmed -- TEDx, Cancelled):`

```
{
  "ticketTier": "VIP",
  "groupSize": 1,
  "notes": "Scheduling conflict"
}
```

15.3.2 `booking_items` table

id	booking_id	line	session_title	qty	unit_price	status
1	1	1	Opening Night	2	1000.00	CONFIRMED
2	1	2	Day 2 — Jazz Fusion	2	500.00	CONFIRMED
3	2	1	Match Day	3	200.00	RESERVED
4	3	1	Morning Talks	1	1000.00	REFUNDED

(line = lineNumber, qty = quantity, sessionId omitted for space)

JSONB column — `metadata`:

Item 1 (Opening Night VIP x2):

```
{
  "seatSection": "VIP Front",
  "rowNumber": "A",
  "ticketTier": "VIP",
  "attendeeNames": ["Ahmed Hassan", "Nour Hassan"]
}
```

Item 3 (Match Day Standard x3):

```
{
  "seatSection": "East Stand",
  "rowNumber": "R12",
  "ticketTier": "standard",
  "attendeeNames": ["Sara Mohamed", "Ali Mohamed", "Layla Mohamed"]
}
```

Reading the data: Booking #1 was made by Ahmed for the Cairo Jazz Festival — he booked 2 VIP tickets for Opening Night (2000 EGP) and 2 tickets for Day 2 (1000 EGP), total 3000 EGP. The booking is completed. Booking #2 is Sara's family outing to the football match — 3 standard tickets at 200 EGP each. Booking #3 was Ahmed's TEDx booking but he cancelled due to a scheduling conflict.

15.4 Ticket Service Tables

15.4.1 `tickets` table

id	booking_id	attendee_name	ticket_code	status	issued_at
1	1	Ahmed Hassan	TIX-2026-001	USED	2026-03-10 09:10
2	1	Nour Hassan	TIX-2026-002	USED	2026-03-10 09:10
3	2	Sara Mohamed	TIX-2026-003	VALID	2026-03-18 15:15
4	2	Ali Mohamed	TIX-2026-004	VALID	2026-03-18 15:15
5	2	Layla Mohamed	TIX-2026-005	VALID	2026-03-18 15:15

JSONB column — `metadata`:

Ticket 1 (Ahmed -- Jazz VIP):

```
{
  "seatNumber": "A1",
  "section": "VIP Front",
  "tier": "VIP",
  "gate": "Gate 1",
  "qrCodeUrl": "https://tickets.example.com/qr/TIX-2026-001",
  "checkInTime": "2026-03-15T19:30"
}
```

Ticket 3 (Sara -- Football Standard):

```
{
  "seatNumber": "R12-S5",
  "section": "East Stand",
  "tier": "standard",
  "gate": "Gate 7",
  "qrCodeUrl": "https://tickets.example.com/qr/TIX-2026-003",
  "checkInTime": null
}
```

Reading the data: Ahmed and Nour attended the Jazz Festival — both tickets are USED with check-in

timestamps. Sara's family has 3 VALID tickets for the upcoming football match (no check-in yet). There are no tickets for Booking #3 because it was cancelled before tickets were issued.

15.5 Sales Service Tables

15.5.1 `ticket_sales` table

id	booking_id	user_id	amount	method	status	created_at
1	1	1	3000.00	CREDIT_CARD	COMPLETED	2026-03-10 09:15
2	2	2	600.00	WALLET	PENDING	2026-03-18 15:20
3	3	1	1000.00	CREDIT_CARD	REFUNDED	2026-03-05 11:10

JSONB column — `transactionDetails`:

`TicketSale 1 (Ahmed -- Completed):`

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "receiptUrl": "https://pay.example/evt001",
  "bookingReference": "BK-2026-0001"
}
```

`TicketSale 2 (Sara -- Pending):`

```
{
  "gatewayResponse": null,
  "walletBalance": 8000,
  "bookingReference": "BK-2026-0002"
}
```

`TicketSale 3 (Ahmed -- Refunded):`

```
{
  "gatewayResponse": "approved",
  "cardLastFour": "4242",
  "refundReason": "Booking cancelled",
  "refundedAt": "2026-03-05T12:00",
  "bookingReference": "BK-2026-0003"
}
```

15.5.2 `promotions` table

id	code	discount_type	value	max	used	expiry_date	active
1	SHOW25	PERCENTAGE	25.0	100	2	2026-06-30 23:59	true
2	FLAT100	FIXED	100.0	50	1	2026-04-30 23:59	true
3	EARLYBIRD	PERCENTAGE	15.0	20	20	2026-02-28 23:59	false

(Column headers shortened: value = discountValue, max = maxUses, used = currentUses)

JSONB column — `metadata`:

`Promotion 1 (SHOW25):`

```
{
  "eligibleCategories": ["CONCERT", "THEATER"],
  "minimumBooking": 500,
  "termsAndConditions": "Valid for concerts and theater only"
}
```

`Promotion 2 (FLAT100):`

```
{
  "eligibleCategories": ["SPORTS"],
  "minimumBooking": 300,
  "applicableEventIds": [2]
}
```

`Promotion 3 (EARLYBIRD):`

```
{
  "eligibleCategories": ["CONCERT", "SPORTS", "CONFERENCE"],
  "minimumBooking": 200,
  "termsAndConditions": "First 20 bookings only"
}
```

15.5.3 `sale_promotions` table (Join Entity)

id	ticket_sale_id	promotion_id	discount_applied	applied_at
1	1	1	750.00	2026-03-10 09:14
2	2	1	150.00	2026-03-18 15:19
3	2	2	100.00	2026-03-18 15:19

Reading the data:

- TicketSale #1 (Ahmed's Jazz Festival booking, 3000 EGP) had promotion SHOW25 applied — 25% of 3000 = 750 EGP discount.
- TicketSale #2 (Sara's football match booking, 600 EGP) had **two** promotions applied: SHOW25 (25% = 150 EGP) and FLAT100 (fixed 100 EGP). Total discount = 250 EGP.
- TicketSale #3 (Ahmed's cancelled TEDx booking) had no promotions applied.
- Promotion SHOW25 has `current_uses = 2` because it was used in sales #1 and #2.
- Promotion EARLYBIRD is inactive because it has reached its max uses (20/20) and its expiry date has passed.

15.6 How the Tables Connect (Cross-Service)

Since all services share one database, you can trace data across tables:

a) **Ahmed (user 1)** booked **Booking #1** for the **Cairo Jazz Festival** (event 1), reserving 2 VIP tickets for Opening Night and 2 tickets for Day 2.

b) Both tickets (**TIX-2026-001** and **TIX-2026-002**) were issued and used — Ahmed and Nour attended the festival and checked in at Gate 1.

c) Ahmed paid via **TicketSale #1** (CREDIT_CARD, 3000 EGP) with promotion SHOW25 applied (saved 750 EGP).

d) **Sara (user 2)** booked 3 standard tickets for the **Al Ahly vs Zamalek** match. Her tickets are VALID but the match hasn't happened yet, and her payment is still PENDING.

e) A native SQL query in the **Booking Service** can JOIN `bookings` with `users` (user_id), `events` (event_id), `tickets` (booking_id), and `ticket_sales` (booking_id) to build a complete booking history DTO.

This is the kind of cross-service data access you will implement using **native SQL JOINs** on the shared database.